

(Task 1 of 19) In this tutorial, we will learn about *local binding forms*: `let` and `letrec` and `let*`. 0

We will first focus on `let`. The following program *binds* `x` to 2 and `y` to 3, and evaluates `(+ x y)`. The result of running this program is 5.

```
(let ([x 2]
      [y (+ 2 1)])
  (+ x y))
```

In SMoL, `let` binds the *left-hand-side variables* (in this case, `x` and `y`) to the values of the *right-hand-side expressions* (in this case, 2 and `(+ 2 1)`) such that the *body* (in this case, `(+ x y)`) can refer to all those left-hand-side variables.

Furthermore, the bindings are *local*, which means we can't refer to the `let`-bound variables outside the `let`.

For example, the following program produces 4 10.

```
(defvar x 10)
(let ([x 4])
  x)
x
```

(Task 2 of 19) What is the result of running this program? 1

```
(let ([x 2])
  (let ([y 30])
    (+ x y)))
```

32

You predicted the output correctly 🎉🎉🎉 3

(Task 3 of 19) What is the result of running this program? 4

```
(defvar x 3)
(let ([x 5])
  (+ x 1))
(+ x 2)
```

6 5

You predicted the output correctly 🎉🎉🎉 6

You predicted the output correctly! 🎉🎉🎉

This program includes two blocks: the top-level block and the body of the `let` expression. The first addition (i.e., `(+ x 1)`) refers to the local `x`, the `x` defined in the body of the `let` expression. So, it produces the value of `(+ 5 1)`, which is 6. In contrast, the second addition (i.e., `(+ x 2)`) refers to the top-level `x`. So, it produces the value of `(+ 3 2)`, which is 5.

(Task 4 of 19) What is the result of running this program?

```
(defvar x 3)
(defvar y (let ([x 5]) x))
(* x y)
```

15

You predicted the output correctly! 🎉🎉🎉

The top-level `x` is 3. The top-level `y` is bound to the value of `x` in the `let`, which is 5. So, the product is 15.

(Task 5 of 19) A `let` expression binds its left-hand-side variables such that its body is in the scope of all the variables.

(Recall that the scope of a variable is the region of a program where the variable can be referred to.)

Any feedback regarding these statements? Feel free to skip this question.

(You skipped the question.)

(Task 6 of 19) What is the result of running this program?

```
(let (foo 41)
  (+ foo 1))
```

error

You predicted the output correctly! 🎉🎉🎉

This program has a syntax error! It should say `([foo 41])` rather than `(foo 41)`.

(Task 7 of 19) What we have seen so far also applies to `letrec` and `let*`.

16

(Task 8 of 19) What is the result of running this program?

17

```
(let ([x 2]
      [y (+ x 1)])
  (+ x y))
```

error

18

You predicted the output correctly 🎉🎉🎉

19

The program errors because `(+ x 1)` is not in the scope of `x`.

(Task 9 of 19) What is the result of running this program?

20

```
(let ([x 42]
      [getx (lambda () x)])
  (getx))
```

error

21

You predicted the output correctly 🎉🎉🎉

22

The scope of `x` only includes the `let` body (i.e., `(getx)`) and does NOT include any of the right-hand-side expressions (i.e., `42` and the `lambda` expression). So, the `x` in the `lambda` expression is undefined.

(Task 10 of 19) What is the result of running this program?

23

```
(let ([f (lambda (n)
            (or (eq? n 0)
                (f (- n 1)))))])
  (f 3))
```

#t

24

Please briefly explain why you think the answer is `#t`.

25

,	26
The answer is <code>error</code>. ▼ Textual explanation The program has a scope error. The <code>lambda</code> expression is not in the scope of <code>f</code>. So, it can not refer to itself.	27
What is the result of running this program? <pre>(let ([h (lambda (x) (or (eq? x 64) (h (* x 2)))))] (h 2))</pre>	28
<code>error</code>	29
You predicted the output correctly 🎉🎉🎉	30
(Task 11 of 19) What is the scope of a left-hand-side variable of <code>let</code> ?	31
<code>final</code>	32

(Task 12 of 19) In a `let`, the scope of a left-hand-side variable includes *none* of the right-hand-side expressions.³³

If we put it visually,

```
(let ([v1 .....]
      [v2 .....]
      [v3 .....]
      ...)
```

.....

)

Any feedback regarding these statements? Feel free to skip this question.

34

(You skipped the question.)

35

(Task 13 of 19) `letrec` and `let*` are similar to `let`: all of them declare their left-hand-side variables such that their bodies are in the scope of all the left-hand side variables.

36

The following program tries to use `let` to define the factorial function.

```
(let ([f (lambda (n)
           (if (eq? n 0)
               1
               (* (f (- n 1)) n)))]])
  (f 5))
```

This program will NOT produce a value. Recall that in a `let`, the scope of a left-hand-side variable includes none of the right-hand-side expressions. So, the `lambda` expression can NOT refer to `f`.

If we use `letrec` rather than `let`, this program will work.

You can even use `letrec` to define mutually recursive functions. For example,

```
(letrec ([od? (lambda (n)
                (if (eq? n 0)
                    #f
                    (ev? (- n 1))))])
  [ev? (lambda (n)
          (if (eq? n 0)
              #t
              (od? (- n 1))))])
  (od? 2))
```

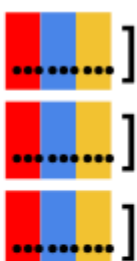
Let's summarize the scoping rules of `letrec`.

(Task 14 of 19) In a `letrec`, the scope of a left-hand-side variable includes *all* of the right-hand-side expressions.

37

If we put it visually,

```
(letrec ([v1
          [v2
          [v3
```





Any feedback regarding these statements? Feel free to skip this question.

38

(You skipped the question.)

39

(Task 15 of 19) The following program tries to use `let` to process `x` through a series of operations.

40

```
(defun (double n)
  (* n 2))
(defun (add1 n)
  (+ n 1))

(let ([x 0]
      [x (add1 x)]
      [x (double x)])
  (+ 3 x))
```

This program will error because none of the right-hand-side expressions can refer to `x`. Replacing `let` with `letrec` will NOT fix the error. Recall that in a `letrec`, the scope of a left-hand-side variable includes *all* of the right-hand-side expressions. To achieve this while avoiding confusion, `letrec` insists that all left-hand-side variables must be distinct. (`let` also has this restriction, but for a different reason.)

If we replace `let` with `let*`, this program will produce 5, the value of `(+ 3 (double (add1 0)))`. `let*` allows you to do a series of bindings sequentially.

Let's summarize the scoping rules of `let*`. After that, we will do a handful of exercises to test your understanding of `let`, `letrec`, and `let*`.

(Task 16 of 19) In a `let*`, the scope of a left-hand-side variable includes *all of the subsequent* right-hand-side expressions.

41

If we put it visually,

```
(let* ([v1 .....]
      [v2 .....]
      [v3 .....])
  ...)
```



Any feedback regarding these statements? Feel free to skip this question.

42

(You skipped the question.)

43

(Task 17 of 19) What is the result of running this program?

44

```
(defvar x 1)
(defvar y 2)

(let ([x (+ y 1)]
      [y (+ x 1)])
  (mvec x y))
```

error

45

The answer is #(3 2).

46

► Textual explanation

What is the result of running this program?

47

```
(defvar a 1)
(defvar b 2)

(let ([a (* b 10)]
      [b (* a 10)])
  (mvec a b))
```

#(20 10)

48

You predicted the output correctly 🎉🎉🎉

49

(Task 18 of 19) What is the result of running this program?

50

```
(defvar x 1)
```

```
(defvar x 1)
(defvar y 2)

(letrec ([x (+ y 1)]
         [y (+ x 1)])
  (mvec x y))
```

error

51

You predicted the output correctly 🎉🎉🎉

52

`(+ y 1)` refers to the local `y`, which is not bound to a value (but a 💣) when `(+ y 1)` is evaluated.

(Task 19 of 19) What is the result of running this program?

53

```
(defvar x 1)
(defvar y 2)

(let* ([x (+ y 1)]
       [y (+ x 1)])
  (mvec x y))
```

#(3 4)

54

You predicted the output correctly 🎉🎉🎉

55

The top-level environment binds `x` to 2 and `y` to 3.

`(+ y 1)` refers to the top-level `y` while `(+ x 1)` refers to the local `x`.

Let's review what we have learned in this tutorial.

56

A `let` expression binds its left-hand-side variables such that its body is in the scope of all the variables.

(Recall that the scope of a variable is the region of a program where the variable can be referred to.)

In a `let`, the scope of a left-hand-side variable includes *none* of the right-hand-side expressions.

If we put it visually,


```
(let ([v1 .....]
      [v2 .....]
      [v3 .....]
      ...)
```



In a `letrec`, the scope of a left-hand-side variable includes *all* of the right-hand-side expressions.

If we put it visually,

```
(letrec ([v1  .....]
         [v2  .....]
         [v3  .....]
         ...)
```



In a `let*`, the scope of a left-hand-side variable includes *all of the subsequent* right-hand-side expressions.

If we put it visually,

```
(let* ([v1 .....]
       [v2  .....]
       [v3  .....]
       ...)
```



You have finished this tutorial 🎉🎉🎉

Please the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1781423573379